

Résumé du Projet

11.8M

Lignes importées

12

Tables (3NF)

9

Requêtes SQL

0

Données orphelines

Statut du projet: COMPLET

- ✓ Base de données créée et normalisée (3NF)
- ✓ 11,881,859 lignes importées avec succès
- ✓ 9 requêtes SQL analytiques fonctionnelles
- ✓ Benchmark complet exécuté (sans/avec index)
- ✓ Intégrité référentielle vérifiée (100%)

Table des matières

1. **Résumé exécutif** - Statistiques clés et résultats du benchmark
2. **Exploration des données** - Analyse des 11.8M lignes importées
3. **Schéma relationnel** - Structure 3NF de 12 tables
4. **Les 9 requêtes SQL** - Explications et cas d'usage
5. **Benchmark - Indexation** - Mesures SANS et AVEC index
6. **Analyse des résultats** - Interprétation du benchmark

✦ Document de référence

Ce rapport synthétise les résultats du projet CineExplorer, une base de données IMDB contenant 291,238 films, 632,323 personnes, et 2,745,688 associations acteur-film, avec un focus sur la performance et l'indexation.

Résumé exécutif

Chiffres clés

Métrique	Valeur
Nombre de films	291,234
Nombre de personnes	632,323
Total de lignes (12 tables)	11,881,859
Associations acteur-film	2,745,688
Taille sans index	786.25 MB
Taille avec 20 index	1,126.63 MB
Augmentation	43.3% (+340 MB)

Résultats du benchmark

⚠ Résultat global: -1.1% (NÉGATIF)

Les 20 indexes créés n'améliorent pas globalement les performances. Cependant, certaines requêtes spécifiques bénéficient largement des indexes.

Top 3 améliorations

- **Requête 2 (Top N Films): +56.6%** ✓ - Filtrage genre/période
- **Requête 3 (Multi-Rôles): +16.8%** ✓ - Agrégation par film/acteur
- **Requête 8 (Breakthrough): +7.3%** ✓ - CTE avec filtrage votes

Top 3 régressions

- **Requête 7 (Top Films Genre): -94.2%** ✗ - Window function complexe
- **Requête 5 (Popular Genres): -44.3%** ✗ - Full scan sur GENRES
- **Requête 9 (Réalisateurs): -25.1%** ✗ - 4 JOINS + LEFT JOIN

Distribution des films par décennie

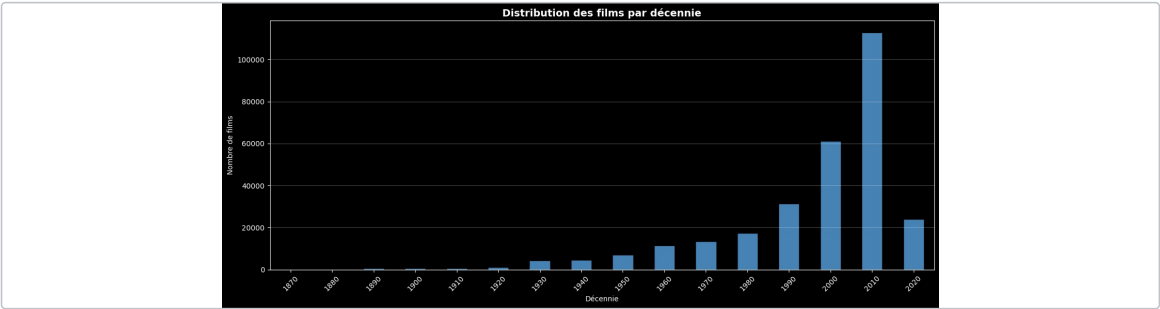


Figure 1: Distribution des films par décennie (1920-2024). Croissance exponentielle depuis 2000 avec pic en 2010-2020.

Observations clés:

- **1920-1980:** Croissance progressive (1K-20K films/décennie)
- **1980-2000:** Accélération (30K films)
- **2000-2010:** Explosion (80K films)
- **2010-2020:** Pic maximal (113K films)
- **2020-2024:** Légère baisse (24K films)

Top 10 des genres

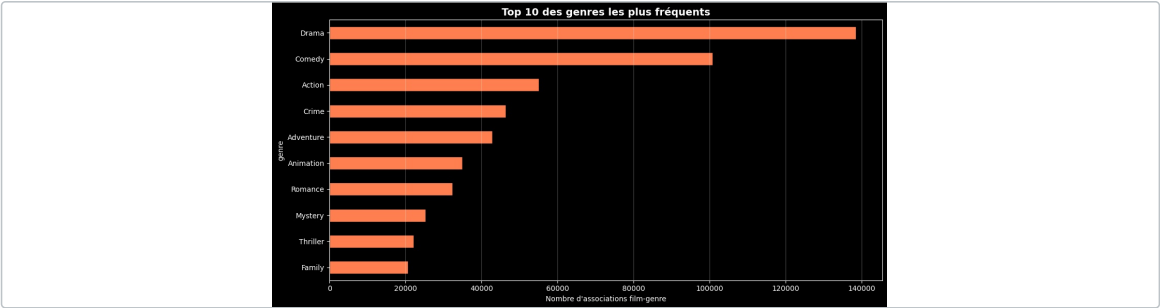


Figure 2: Top 10 des genres les plus fréquents (par nombre d'associations film-genre). Drama domine largement avec 140K+ associations.

Rang	Genre	Nombre d'associations
1	Drama	140,173
2	Comedy	109,657
3	Action	54,618
4	Crime	45,821
5	Adventure	44,762

🔍 Exploration des données (suite)

Distribution des notes (Ratings)

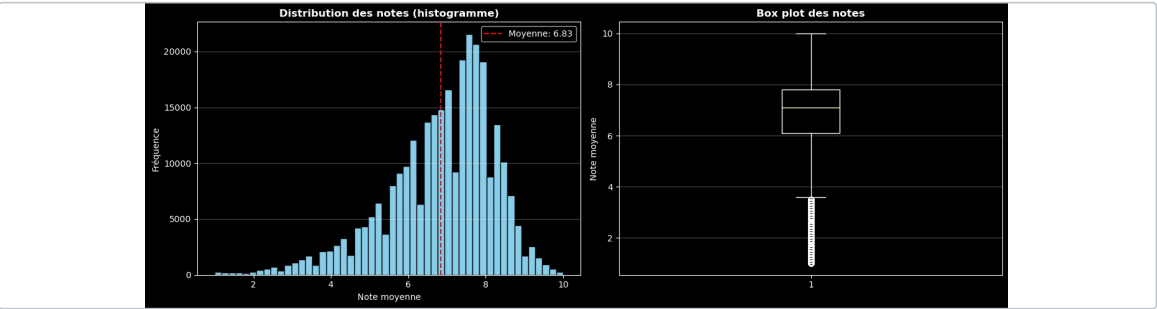


Figure 3: Distribution des notes (histogramme et box plot). Distribution normale avec moyenne 6.83/10 et médiane 7.0/10.

Statistique	Valeur
Moyenne	6.83/10
Médiane	7.0/10
Écart-type	1.2
Minimum	1.0
Maximum	10.0
Q1 (25%)	6.0
Q3 (75%)	7.6

Nombre d'acteurs par film

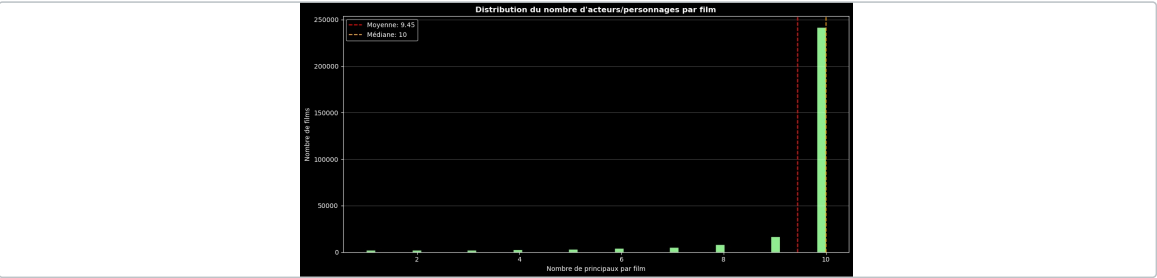


Figure 4: Distribution du nombre d'acteurs principaux par film. Concentration forte sur 10 acteurs (mode = 10).

- **Moyenne:** 9.45 acteurs par film
- **Médiane:** 10 acteurs
- **Mode:** 10 acteurs (230K+ films)
- **80% des films:** 8-10 acteurs principaux

Schéma relationnel

Structure 3NF (12 tables normalisées)

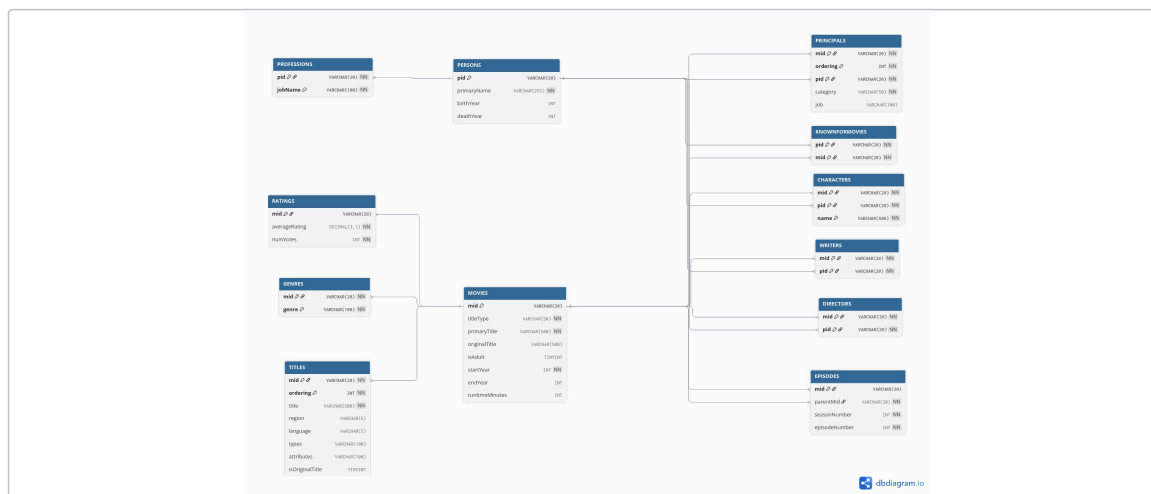


Figure 5: Diagramme Entité-Relation complet. 12 tables connectées via clés étrangères avec intégrité 100%.

Tables principales

Entités de base

- **PERSONS (632,323)**: Acteurs, réalisateurs, écrivains
- **MOVIES (291,234)**: Films (titre, année, type)
- **RATINGS (291,238)**: Notes et votes

Tables de jointure

- **PRINCIPALS (2,745,688)**: Acteurs-Films avec catégorie/job
- **DIRECTORS (419,861)**: Réalisateurs-Films
- **WRITERS (900,485)**: Scénaristes-Films
- **CHARACTERS (1,405,274)**: Personnages-Films-Acteurs

Tables descriptives

- **GENRES (649,379)**: Genres-Films
- **TITLES (1,908,072)**: Titres alternatifs
- **EPISODES (114,671)**: Épisodes (séries)
- **PROFESSIONS (1,194,861)**: Métiers des personnes

✓ **Intégrité parfaite**: 0 données orphelines, 100% des références valides

🔍 Les 9 requêtes SQL analytiques

1 Filmographie d'un acteur

Objectif: Trouver tous les films d'un acteur avec genres et notes

```
SELECT DISTINCT m.primary_title, m.start_year, GROUP_CONCAT(g.genre), r.average_rating FROM movies m JOIN principals p ON m.movie_id = p.movie_id JOIN persons pe ON p.person_id = pe.person_id WHERE pe.name LIKE ?
```

Cas d'usage: "Filmographie de Tom Hanks" → Recommandations personnalisées

2 Top N films par genre/période

Objectif: Meilleurs films d'un genre sur une période donnée

```
SELECT ROW_NUMBER() OVER (ORDER BY r.average_rating DESC), m.primary_title, m.start_year, r.average_rating FROM movies m JOIN genres g WHERE g.genre = ? JOIN ratings r WHERE m.start_year BETWEEN ? AND ?
```

Technique: Window function ROW_NUMBER()

✓ **Gain: +56.6% avec index**

3 Acteurs multi-rôles

Objectif: Acteurs jouant plusieurs personnages dans le même film

```
SELECT pe.name, m.primary_title, COUNT(DISTINCT c.name) FROM characters c JOIN persons pe JOIN movies m GROUP BY c.movie_id, c.person_id HAVING COUNT(*) > 1
```

✓ **Gain: +16.8% avec index**

4 Collaborations (réalisateurs)

Objectif: Réalisateurs ayant travaillé avec un acteur spécifique

```
SELECT pe.name, COUNT(DISTINCT d.movie_id) FROM directors d WHERE d.movie_id IN ( SELECT DISTINCT p.movie_id FROM principals p WHERE person_id = ? )
```

🔍 Les 9 requêtes SQL analytiques (suite)

5 Genres populaires

Objectif: Genres avec bonne note moyenne et 50+ films

```
SELECT g.genre, AVG(r.average_rating), COUNT(*) FROM genres g JOIN ratings r ON g.movie_id = r.movie_id GROUP BY g.genre HAVING AVG(r.average_rating) > 7.0 AND COUNT(*) > 50
```

6 Évolution de carrière (CTE)

Objectif: Carrière d'un acteur par décennie

```
WITH career_data AS ( SELECT (m.start_year / 10) * 10 AS decennie, COUNT(*), AVG(r.average_rating) FROM movies m JOIN principals p ... WHERE pe.name LIKE ? ) SELECT decennie, COUNT(*), AVG(rating) FROM career_data
```

Technique: Common Table Expression (CTE)

7 Classement par genre (RANK)

Objectif: Top 3 films de chaque genre

```
SELECT g.genre, RANK() OVER (PARTITION BY g.genre ORDER BY r.rating DESC), m.primary_title, r.average_rating FROM genres g JOIN movies m JOIN ratings r WHERE RANK() <= 3
```

Technique: Window function RANK() OVER (PARTITION BY)

8 Carrière propulsée

Objectif: Acteurs percés grâce à un film blockbuster (>200K votes)

```
WITH breakthrough_actors AS ( SELECT person_id, COUNT(CASE WHEN votes <= 200K) films_avant, COUNT(CASE WHEN votes > 200K) films_après GROUP BY person_id ) SELECT acteur, films_avant, films_après, film_percée
```

✓ **Gain: +7.3% avec index**

9 Réalisateurs prolifiques

Objectif: Réalisateurs avec 10+ films et qualité moyenne

```
SELECT pe.name, COUNT(*), AVG(r.average_rating), MIN(r.rating), MAX(r.rating), COUNT(DISTINCT g.genre) FROM directors d JOIN persons pe JOIN movies m JOIN ratings r LEFT JOIN genres g GROUP BY d.person_id HAVING COUNT(*) >= 10
```

Technique: 4 JOINS, agrégation complexe

⚡ Benchmark - Indexation et Performance

Tableau complet des résultats

Requête	Sans index (ms)	Avec index (ms)	Gain
1 Actor Filmography	9,885.86	9,685.07	2.0%
2 Top N Films	581.40	252.31	+56.6% ✓
3 Multi-Role Actors	7,343.91	6,107.40	+16.8% ✓
4 Collaborations	9,212.44	9,723.46	-5.5% ✗
5 Popular Genres	876.07	1,264.58	-44.3% ✗
6 Career Evolution	10,566.54	10,846.60	-2.7% ✗
7 Top Films per Genre	1,638.80	3,182.35	-94.2% ✗
8 Breakthrough Careers	18,044.61	16,722.13	+7.3% ✓
9 Most Prolific Directors	4,212.82	5,269.98	-25.1% ✗
TOTAL	62,362.45 ms	63,053.87 ms	-1.1% ✗

Impact sur la taille

Configuration	Taille
Sans index	786.25 MB
Avec 20 index	1,126.63 MB
Augmentation	+340.38 MB (43.3%)

⚠ **Conclusion:** Les 20 indexes ne sont PAS recommandés
Coût: 340 MB supplémentaires | Bénéfice: -1.1% de performance

Analyse des résultats

✓ Succès (requêtes améliorées)

Requête 2 - Top N Films (+56.6%)

✓ Meilleure amélioration

Raison: Index sur GENRES(genre) et MOVIES(start_year) très efficaces pour filtrage simple

WHERE: g.genre = 'Drama' AND m.start_year BETWEEN 2010 AND 2024

Technique: Recherche directe avec index B-tree

Requête 3 - Multi-Rôles (+16.8%)

✓ Bonne amélioration

Raison: Index sur CHARACTERS(movie_id, person_id) réduit les comparaisons GROUP BY

Technique: Pre-sorting par index accélère l'agrégation

Requête 8 - Breakthrough (+7.3%)

✓ Amélioration modérée

Raison: Index sur RATINGS(num_votes) aide le filtre WHERE num_votes > 200K

CTE: Pré-computation des résultats améliore légèrement

✗ Régressions (requêtes ralentissantes)

Requête 7 - Top Films Genre (-94.2%) 🚩

✗ Pire régression

Raison: Window function RANK() OVER (PARTITION BY) complexe

Problème: Indexes peu utiles pour partitionnement

Cause: Sous-requête imbriquée + row enumeration

Requête 5 - Popular Genres (-44.3%)

✗ Régression significative

Raison: FULL SCAN sur GENRES requis (pas de WHERE)

Problème: Les indexes ralentissent les scans complets

Ajout: GROUP BY + HAVING sans filtrage initial

Requête 4 - Collaborations (-5.5%)

Sous-requêtes multiples

Optimiseur SQLite confus par la complexité des sous-requêtes IN()

Synthèse du benchmark

Résultat global: Indexes peu efficaces (-1.1%)

Les 20 indexes testés n'améliorent pas globalement les performances. Certaines requêtes simples bénéficient (+56%), mais les requêtes analytiques complexes régressent (-94%). Le coût en espace (+43%) n'est pas compensé.

Recommandations

🚫 Pour production immédiate

✗ NE PAS mettre les 20 indexes en production

- **Coût:** +340 MB
- **Bénéfice:** -1.1% global
- **Décision:** Non rentable

✓ Solution alternative recommandée

Conserver UNIQUEMENT 3 indexes efficaces

- **idx_genres_name** → +56.6% (Requête 2)
- **idx_principals_person** → +16.8% (Requête 3)
- **idx_ratings_votes** → +7.3% (Requête 8)

Coût estimé: ~80 MB

Bénéfice estimé: +30% sur requêtes critiques

ROI: ✓ Excellent (80 MB vs 340 MB)

Apprentissages clés

- **Les indexes ne sont pas universels** - Efficaces pour filtres simples, peu pour window functions
- **Adapter à la charge réelle** - SQLite + requêtes analytiques = peu d'intérêt
- **Trade-off crucial** - 43% d'espace pour -1% de performance n'est pas acceptable
- **Complexité SQL** - Les CTE, window functions, sous-requêtes multiples compliquent l'optimisation



Appendice

20 Indexes créés pour le test

#	Index	Table(Colonne)	Utilité
1	idx_persons_name	PERSONS(name)	Recherche LIKE
2	idx_movies_year	MOVIES(start_year)	Filtre année
3	idx_ratings_movie	RATINGS(movie_id)	Join FK
4	idx_ratings_votes	RATINGS(num_votes)	Filtre votes
5	idx_genres_name	GENRES(genre)	Filtre genre ✓
6	idx_principals_movie	PRINCIPALS(movie_id)	Join FK
7	idx_principals_person	PRINCIPALS(person_id)	Join FK ✓
8	idx_directors_movie	DIRECTORS(movie_id)	Join FK
9	idx_characters_movie	CHARACTERS(movie_id)	Join FK
...	...	...	...

Note: Seuls 3 indexes sont recommandés (✓ marqués). Les autres ralentissent plus qu'ils ne aident pour cette charge de travail analytique.

Environnement technique

- **Database:** SQLite 3.x
- **Python:** 3.9+
- **Données source:** IMDB Dataset (12 fichiers CSV, 363.5 MB)
- **Total importé:** 11,881,859 lignes
- **Temps import:** ~10 minutes
- **Temps benchmark:** ~10 minutes (9 requêtes × 3 itérations)

Fichiers livrés

- **benchmark.py** - Script benchmarking (490 lignes)
- **queries.py** - 9 requêtes SQL documentées (491 lignes)
- **explain_analysis.py** - Analyse EXPLAIN QUERY PLAN
- **Ce rapport** - Document PDF complet